

마이크로프로세서개론: 프로그래밍과제 #1

20181501 김준원

1. 목적 및 고려사항

✓ 목적

Assembly language를 이용한 programming을 통해 프로세서의 동작을 프로그램 수행과정에서 register들의 역할, 메모리에 저장된 변수의 access 방법, stack의 역할 등을 통해 구체적으로 이해할 수 있는 기회를 갖는다. 또한 modular programming에 대해 이해한다.

✓ 고려사항

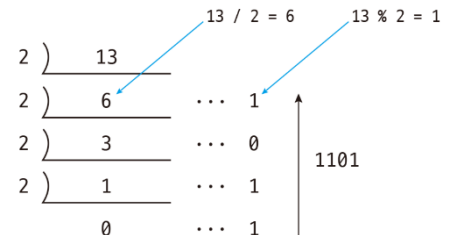
- Subroutine의 역할 및 입출력 설정

해당 프로그램의 두 정수에 대해 합을 계산 후 이를 ascii code의 배열로 저장하는 동작을 수행하며, 이 중 계산된 합을 ascii code로 변환 후 저장하는 동작을 subroutine을 통해 구현하고자 하였다.

따라서 기본적으로 Subroutine에는 2진수로 저장된 정수 값, string으로 변환할 문자열의 진수가 기본적으로 제공하였으며, 동작결과를 반환하는 대신 ascii code가 저장될 주소를 통해 간접적으로 값을 반환하도록 작성하였다. 이를 통해 10진수 이외의 진수로의 표현 및 좀 더 큰 정수에 대한 string 반환 등의 프로그램의 확장이 가능하도록 하였다.

- Leading zero 및 문자열 범위

진수를 변환하는 과정은 아래 그림과 같이 정수를 원하는 진수로 나누며 생기는 나머지의 역순이 된다. 나머지를 stack에 저장하여 나머지의 역순을 구하도록 코드를 구성하였으며, 이 때 Stack에 고정된 수의 나머지를 저장하는 것이 아닌, Stack에 저장한 나머지의 길이를 별도의 레지스터에 저장하는 방식을 통해 leading zero 문제를 해결하였다.



2. 코드 분석

__main procedure (main.s)				
...			...	
4		export	__main [weak]	
5		;-----insert your code between the lines-----		
6		extern	itoa	
7		extern	atoi	
8		; itoa subroutine		
9	start	ldrh	r1, num1	; load num1
10		ldrh	r2, num2	; load num2
11		ldr	r0, = sum	; address of sum

12		add	r3, r1, r2	; adding num1 & num2
13		str	r3, [r0]	; storing sum
14		ldr	r0, = sumascii	; store address
15		mov	r4, #10	; base select
16		push	{r0,r3,r4}	; passing parameter
17		bl	itoa	; call subroutine
18		add	sp, #12	; SP initialization
...				...
25		;-----		
21	here	b	here	
22			endp	
23			align	
24	num1	dcw	999	
25	num2	dcw	2	
26			align	
27		area	projdata, data	
28	sum	dcd	0	
29	sumascii	space	4	
30			end	

코드 영역에 Numeric Constant로 선언된 num1과 num2를 load 후(Line 9-10) 이 두 수의 합을 R3 및 data영역의 변수 sum에 저장하였다(Line 12-13). 이후 Line 16에서 sumascii 변수의 시작 주소를 저장한 R0,정수 값 R3, 그리고 진수를 저장하는 R4를 Stack영역에 저장 후 Subroutine 을 호출하여 Subroutine itoa에 parameter를 전달하였다. 이후, SP를 해당 파라미터 개수에 해당하는 #12만큼 올려주어 Stack을 원위치하였다.

subroutine (itoa.s)				
1		area	subcode, code	
2		entry		
3	itoa	proc		
4		export	itoa	
5		;-----		
6		push	{r0-r5,lr}	
7		ldr	r0, [sp, #28]	; store address
8		ldr	r1, [sp, #32]	; integer
9		ldr	r2, [sp, #36]	; base
10		mov	r5, #0	; count
11		; calculating conversion		
12	lp1	udiv	r3, r1, r2	; quotient
13		mls	r4, r2, r3, r1	; remainder
14		push	{r4}	; storing remainder into stack
15		cmp	r3, #0	; check whether quotient == 0
16		add	r5, #1	; increase count
17		mov	r1, r3	; update dividend
18		bne	lp1	; iterate if quotient != 0
19		; initializing location		
20		str	r3, [r0]	
21		; storing ascii code		
22		ldr	r1, =ascii	
23	lp2	pop	{r4}	; load remainder (decimal digit)

24		subs	r5, #1	; decrease count
25		ldrb	r4, [r1, r4]	; convert to ascii code
26		strb	r4, [r0], #1	; store ascii code
27		bne	lp2	
28				; return
29	return	pop	{r0-r5,pc}	
30				;-----
31		endp		
32		align		
33	ascii	dcb	0x30	;'0'
34		dcb	0x31	;'1'
35		dcb	0x32	;'2'
36		dcb	0x33	;'3'
37		dcb	0x34	;'4'
38		dcb	0x35	;'5'
39		dcb	0x36	;'6'
40		dcb	0x37	;'7'
41		dcb	0x38	;'8'
42		dcb	0x39	;'9'
43		dcb	0x41	;'A'
44		dcb	0x42	;'B'
45		dcb	0x43	;'C'
46		dcb	0x44	;'D'
47		dcb	0x45	;'E'
48		dcb	0x46	;'F'
49		end		

해당 Subroutine에서 사용할 레지스터인 R0-R5, 그리고 LR의 값을 Stack에 저장하여, Context 및 복귀 주소를 유지한 후(Line 6) stack에 저장된 parameter를 레지스터에 로드한다(Line7-9). 이후 몫이 0이 될 때까지 나눗셈을 수행하며 나머지를 stack에 저장한다. 이때, R5 레지스터에 나머지의 개수를 저장하게 된다. 첫 번째 loop가 끝나면 두 번째 loop에서는 R5에 저장된 횟수만큼 반복하며 stack에 저장된 나머지를 Ascii code로 변환 후 이를 메모리에 저장한다. Line 33-38은 연속적인 데이터 공간에 각 숫자 digit을 나타내는 ascii 코드를 정의한 것으로 해당 주소에서 ascii code를 로드하여 자릿수를 ascii code로 변환한다(Line 25)

3. 수행 결과

프로그램 수행 결과 #23과 #100의 합에 해당하는 #123이 "123"의 Ascii code로 저장되었음을 확인하였다.

0x20000000: 7B 00 00 00 31 32 33 00

4. 검토사항

1. Sum에 저장된 내용을 10진수가 아닌 16진수 string으로 표현하여 sumascii에 저장한다면 자신이 작성한 프로그램의 어떤 부분을 어떻게 수정해야 하는지 설명.

Sum에 저장된 내용을 16진법의 string으로 저장하기 위해서 우선 각 자릿수를 16진수에 맞게끔 계산 후 이에 맞게 '0'-'9'와 'A'-'F'의 Ascii code로 변환해야 한다. 프로그램에서 base를 저장하는 R4에 #10이 아닌 #16을 저장 후 parameter를 넘겨주면, 프로그램은 정수를 16으로 나눈 나머지를 itoa.s의 line 33-48의 데이터에 맞게 대조(LUT)하여 얻은 ascii code를 메모리 상에 저장하게 된다. 두 정수 num1, num2에 동일하게 #23, #100의 정수를 저장 후 itoa subroutine에 전달되는 base parameter를 전달한 결과 아래와 같이 "7B"의 ascii 코드가 저장됨을 확인하였다.

0x20000000: 7B 00 00 00 37 42 00 00

- 만일 num1과 num2에 string 형태로 10진수가 제공된다면 자신이 작성한 프로그램의 어느 부분이 어떻게 수정되어야 하는지 설명.

2진수를 사용하는 컴퓨터 체계에서 16진수, 8진수를 다른 진수로 변환하는 경우, itoa subroutine에서 사용한 것처럼 정수를 진수로 나누는 방식을 사용해야 한다. 반면, 다른 진수체계의 수의 값을 16진수 혹은 8진수로 나타내는 방법은 이와 반대로 곱하는 방식을 사용한다. 즉, 변환하고자 하는 수의 진수가 b라고 할 시, $d_k d_{k-1} \dots d_1 d_0$ 로 표현되는 수의 값은 다음과 같다. 따라서 string으로 표현된 수는 각 digit의 수와 이에 해당하는 자릿값을 곱한 후 이를 더하면 그 값을 구할 수 있다고 예상할 수 있다.

$$V = d_k \cdot b^k + d_{k-1} \cdot b^{k-1} + \dots + d_1 \cdot b + d_0$$

또한, 메모리 상에서 string이 제공이 될 때, string의 길이를 명시하거나 string의 끝에 특정 문자를 삽입하여 string의 종료를 확인할 수 있다.

프로그램 상에서 atoi subroutine이 string으로 저장된 10진수를 입력 받아 이에 해당하는 정수값을 stack을 통해 반환하는 역할을 수행한다. Main.s에서 itoa subroutine을 통해 저장한 ascii code를 atoi subroutine을 통해 다시 정수형으로 변환하였다. 이때, string의 종료를 확인하기 위해 0x00의 NULL character를 사용하였다.